# CIS*3110 Operating Systems

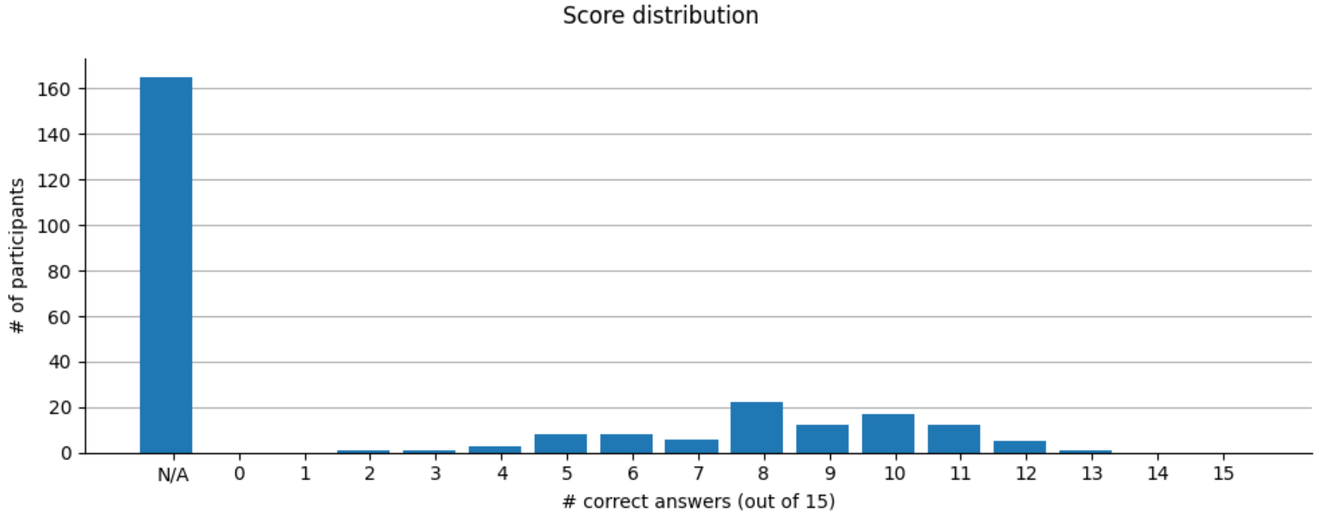
## Week 12
## Lecture #20 FFS

**Prof. Yulia Kotseruba**
**March 26, 2026**

# Quiz 18

```
10-bit virtual address
64KB (2^16) RAM
1KB (2^10) pages
```

- **58%** average (down from 68% in quiz 17)
- 3 hardest questions:
  - (13%) What is the max number of concurrent process using 4 pages each can this system support? No sharing and no swapping. a) 4 b) 16 c) 64 d) none of the above
  - (21%) What is the link count for the "folder" directory below? (-> indicates a symbolic link) a) 4 b) 5 c) 6 d) 7
  - (27%) Deadlock avoidance requires a) rebooting the PC b) rewriting code with locks c) rewriting code without locks d) rescheduling processes




```
folder
|-- file1
|-- subfolder1
|     |-- file1
|-- subfolder2
|-- file3 -> file1
```

# Recap

- Each process tracks its open descriptors
  - per-process file descriptor table
- OS tracks all open file descriptors, their offsets and permissions
  - system-wide open file table linked to inode table
- File descriptors created with open() are **independent** of each other
  - have **different** offset and mode - separate entries in the open file table
  - point to the same inode
- File descriptors created with dup() are a **copy** of another descriptor
  - **share** offset and mode - the same entry in the open file table
  - point to the same inode

# Recap

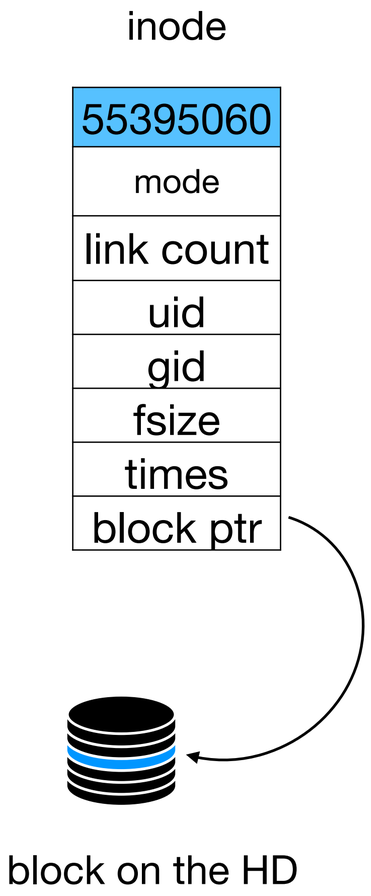
- **Files** - sequences of bytes stored on HD
  - name - human-readable identifier
  - inode - stores file metadata and links to HD
- **Directories** - similar to files
  - name - human-readable identifier
  - inode - stores mapping between files in the directory and their inodes
- Names and contents of files/directories are stored **separately**
  - **Hard links** - different paths for the same inode
  - **Soft (symbolic) links** - different aliases for the same path

# File system implementation

- Data structures
  - how to organize data and metadata in memory?
- Access methods
  - how to use system calls with the data structures?
- very simple file system (vsfs)
  - an abstract version of a typical Unix file system
  - demonstrates a basic file system implementation

# inode

- aka index node
  - the main data structure for storing files
- file inodes
  - file metadata
- directory inodes
  - directory metadata
  - maps names to inodes



# blocks

- Hard drive is divided into blocks of equal size
- Sector size on HD is 512 bytes (rarely 4096 bytes)
- Block size is a multiple of sector size
  - typical size is 4KB
  - ranges from 512 bytes and 64KB depending on the HD and FS

# blocks

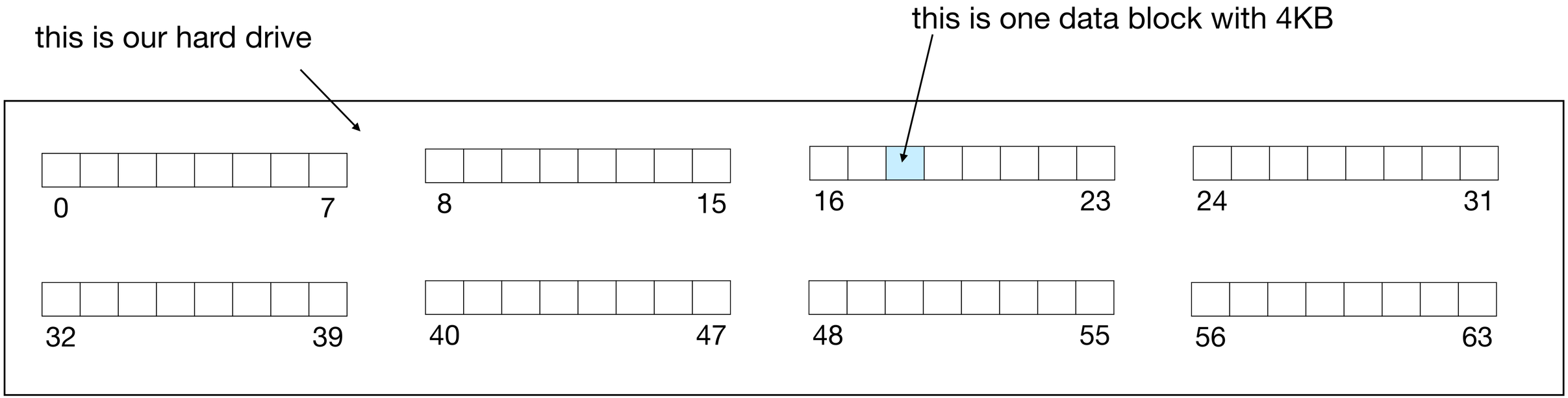
- Assume a (very) small hard drive
  - 4KB block size
  - 64 blocks
  - 256 KB total memory

http://www.kenbakkit.com/

Kenbak-1, 1971
256 bytes
1000 op/s

# blocks

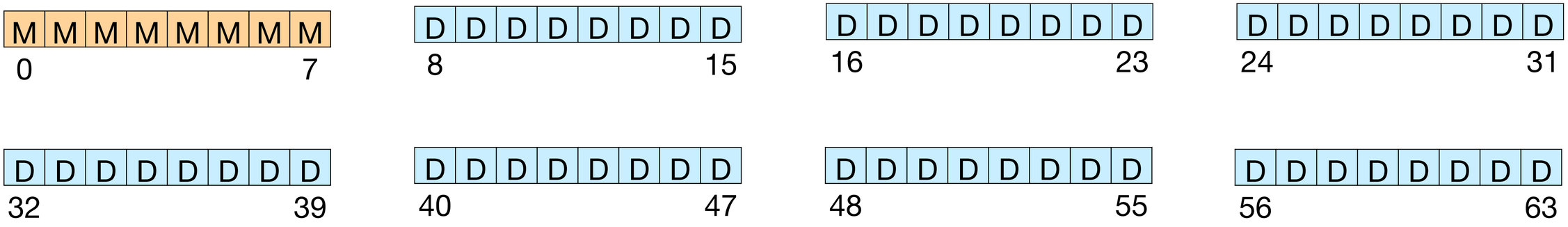
- Assume a (very) small hard drive
  - 4KB block size
  - 64 blocks
  - 256 KB total memory



# blocks

- Assume a (very) small hard drive
  - 4KB block size
  - 64 blocks
  - 256 KB total memory

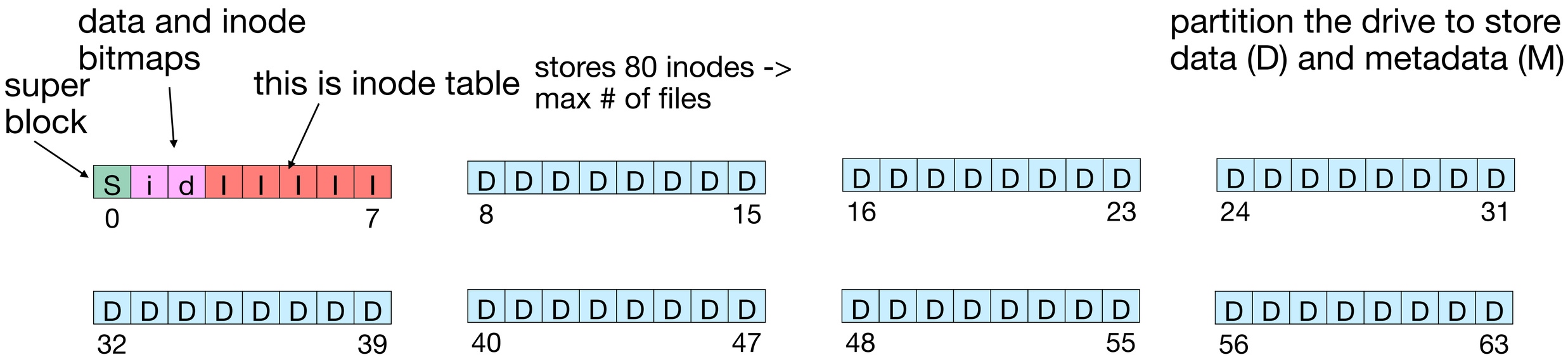
partition the drive to store data (D) and metadata (M)



# blocks

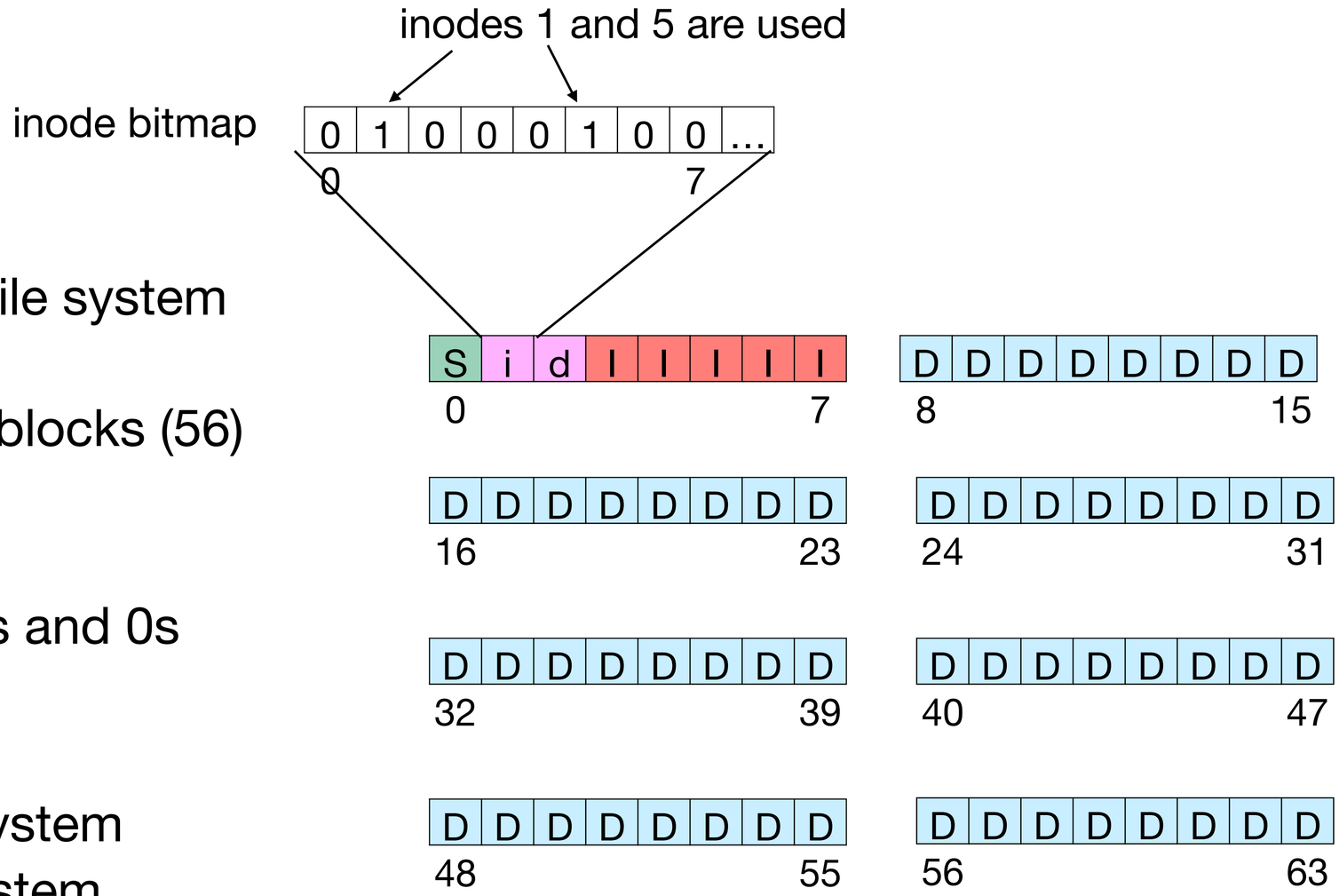
- Assume a (very) small hard drive
  - 4KB block size
  - 64 blocks
  - 256 KB total memory

- Partition the drive
  - 224 KB for data
  - 32 KB for metadata
    - inodes are 256 bytes
    - each 4KB block holds 16 inodes

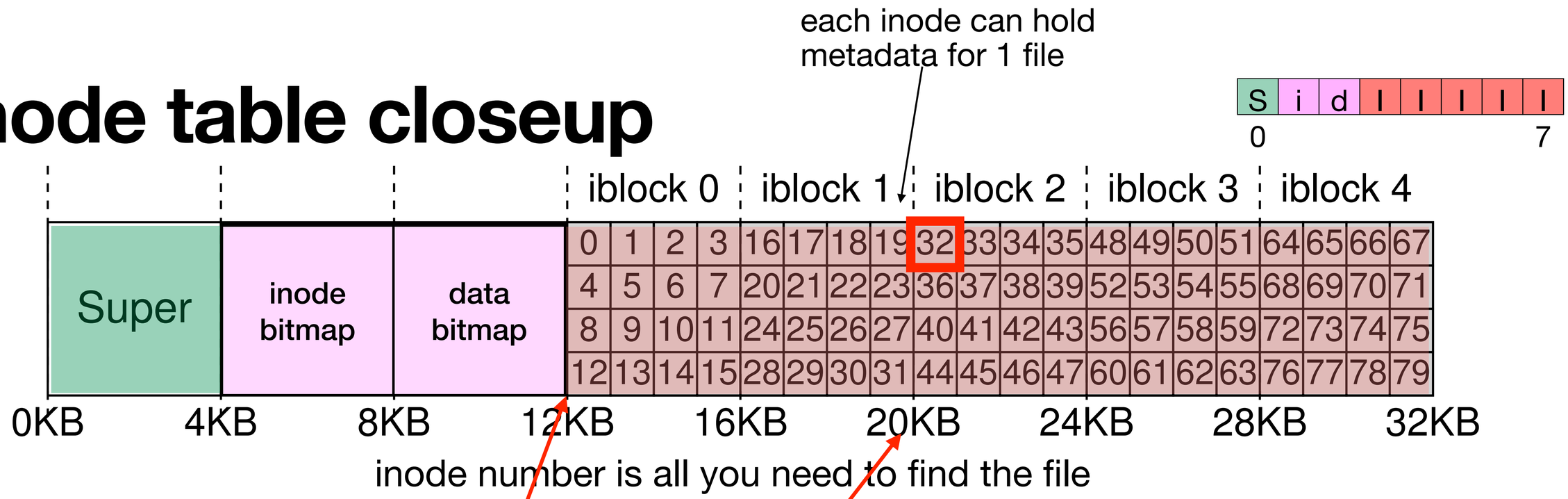


# blocks

- **superblock (S)** - metadata of the file system
  - id of the file system (vsfs)
  - number of inodes (80) and data blocks (56)
  - start of the inode table (block 3)
- **inode (i) and data (d) bitmaps**
  - each bitmap is a 4KB array of 1s and 0s
    - 1 - inode/data block is used
    - 0 - inode/data block is free
- **inode table (I)** - all inodes in the system
  - can store only **80 files** in this system
    - 5 blocks allocated for the table
    - each 4KB block holds 16 inodes
- **data blocks (D)**
  - 4KB in each block



# inode table closeup



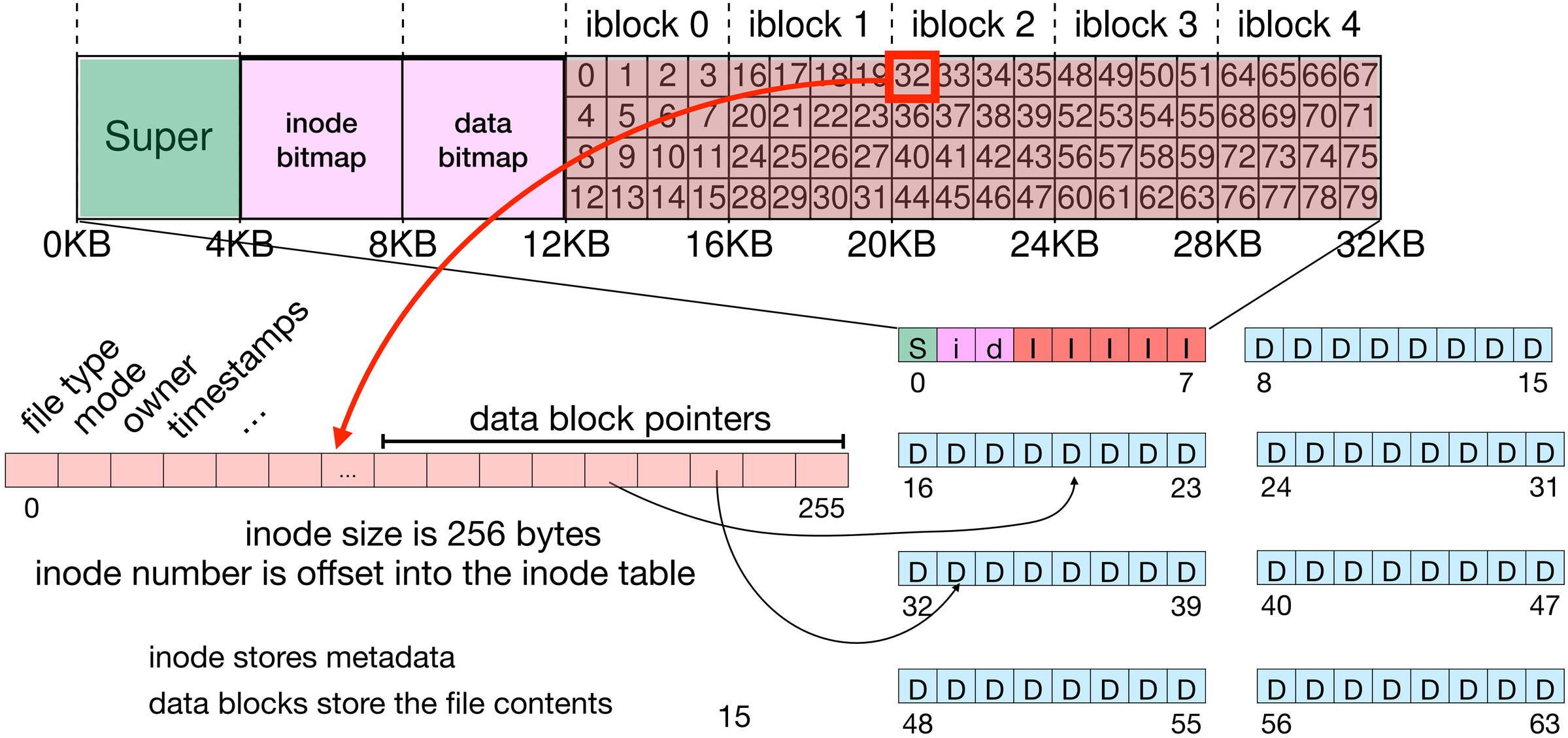

e.g. to access inode #32

- inode table starts at 12KB on the disk
- offset into the inode table: inode id * sizeof(inode) = 32 * 256B = 8,192B = 8KB
- inode address: 12KB + 8KB =20 KB
- disk sector: (20KB)/512B = 40 (each sector is 512 bytes)
  - can issue a read to sector 40 to get the inode

# inode

- aka index node (alternative spelling i-node)
- A data structure that stores **metadata** of the files/directories
  - **inode number** - a unique identifier (offset into the inode table)
  - **file type** - reg file, directory, symbolic link, etc.
  - **permissions** - read, write, execute for user, group, others
  - **owner** - user ID and group ID
  - **size** of the file
  - **timestamps** - time of creation, modification, change, last access
  - **link count** - number of hard links to inode
  - **pointers** - data blocks on the storage device

# inode table closeup




inode size is 256 bytes

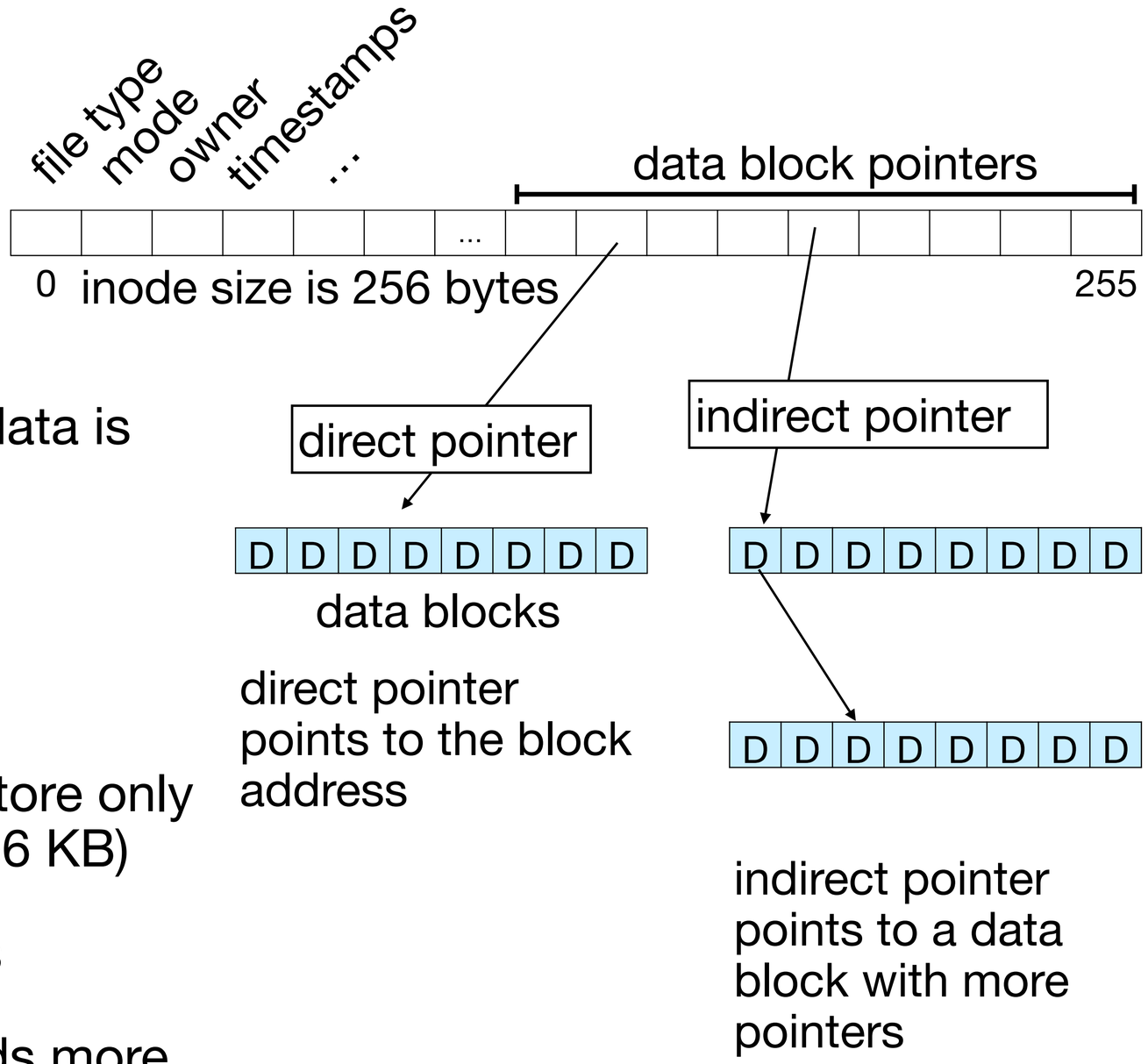
inode number is offset into the inode table

inode stores metadata

data blocks store the file contents

# block pointers

- Direct pointers
  - address of the block where the file data is stored
- Problem #1: what if the file is large?
  - inode size is limited (256 bytes)
  - e.g. if disk address is 4 bytes, can store only 64 block addresses (max file size 256 KB)
- Solution: multi-level indexing of blocks
  - a pointer to a block of data that holds more pointers



# Multi-level block indexing

block size 4KB

pointer size 4 bytes

12 direct pointers

what is the max size of the file that can be stored?

- Each inode contains a **fixed number of block pointers**
  - **direct pointers** - pointer to a data block

    12 * 4KB = 48 KB

  - **1 indirect pointer** - pointer to a block of direct pointers

    (12 + 1024) * 4KB = 4144KB

  - **1 double-indirect pointer** - pointer to a block of pointers to blocks of direct pointers

    $(12 + 1024 + 1024^2)*4KB \sim 4GB$

  - **1 triple-indirect pointer** - you get the idea $(12 + 1024 + 1024^2 + 1024^3)*4KB \sim 4TB$

# Multi-level block indexing

- Why only 3 indirect pointers?
- Most files are small
  - 2KB is the most common size, 200KB average size
- Most filesystems contains many small files
  - 100K files on average
- Most directories are small
  - 20 or fewer files
- Most file systems are 50% full

# Directory inodes

- Directory contains a list of files and directories
  - a linear array with records

folder data is a mapping between inodes and file names

2 hard links to itself and to the parent

inode for /folder

| |
|---|
| 5 |
| mode |
| link count |
| uid |
| gid |
| fsize |
| times |
| block ptr |

self-link to its own inode #

root dir inode is often 2

data for /folder

| inode | record length | string length | name |
|---|---|---|---|
| 5 | 12 | 2 | . |
| 2 | 12 | 3 | .. |
| 12 | 12 | 4 | file1 |
| 13 | 12 | 4 | file2 |
| 24 | 24 | 19 | file_with_long_name |

almost the same as the file inode

record length and name length can vary

# Directory inodes

- Directory contains a list of files and directories
  - a linear array with records

folder data is a mapping between inodes and file names

2 hard links to itself and to the parent

inode for /folder

| |
|---|
| 5 |
| mode |
| link count |
| uid |
| gid |
| fsize |
| times |
| block ptr |

self-link to its own inode #

root dir inode is often 2

data for /folder

| inode | record length | string length | name |
|---|---|---|---|
| 5 | 12 | 2 | . |
| 2 | 12 | 3 | .. |
| ~~12~~ | ~~12~~ | ~~4~~ | ~~file1~~ |
| 13 | 12 | 4 | file2 |
| 24 | 24 | 19 | file_with_long_name |

when a file is deleted (unlinked) its information is removed from the inode, link count is updated

record length and name length can vary

# Directory inodes

- Directory contains a list of files and directories
  - a linear array with records

folder data is a mapping between inodes and file names

2 hard links to itself and to the parent

inode for /folder

| |
|---|
| 5 |
| mode |
| link count |
| uid |
| gid |
| fsize |
| times |
| block ptr |

self-link to its own inode #

root dir inode is often 2

data for /folder

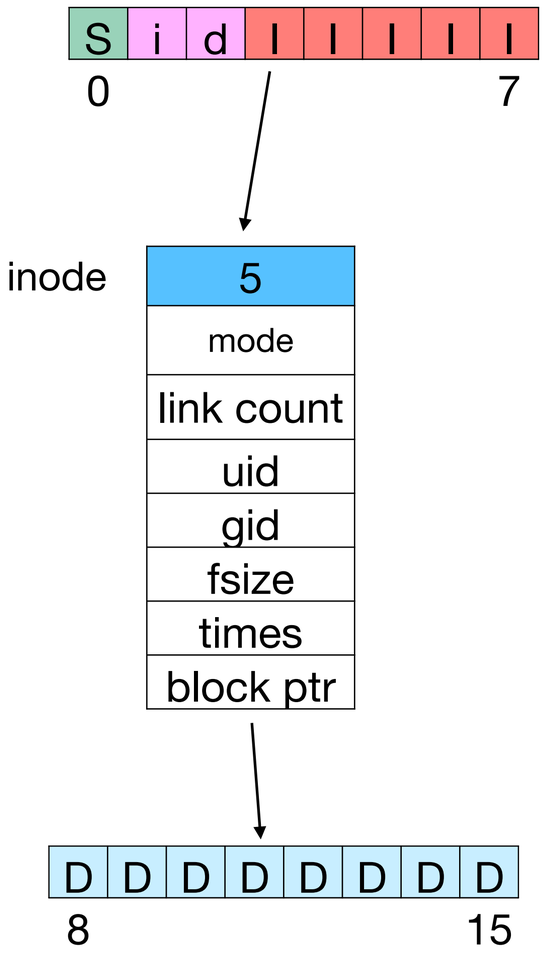
| inode | record length | string length | name |
|---|---|---|---|
| 5 | 12 | 2 | . |
| 2 | 12 | 3 | .. |
| 0 | 12 | | |
| 13 | 12 | 4 | file2 |
| 24 | 24 | 19 | file_with_long_name |

free entries are marked (e.g. with inode 0) and can be reused

record length and name length can vary

# Summary so far

- very simple file system (vsfs)
- **blocks** - fixed sized units of memory on hard drive
  - hold data
- **Metadata structures**
  - **superblock** - metadata about file system
  - **bitmaps** - info about which data blocks and inodes are free
  - **inodes** - contain info for files and directories
    - direct and indirect pointers to data blocks on hard drive



# Access paths

- File path tells the OS how to navigate the system-wide inode table
  - path is split into parts (in Unix-like systems by a / delimiter)
  - OS finds the inode for the first part of the file
  - if it is a directory
    - its inode lists files and directories inside
  - if it is a file
    - its inode will have the link to disk blocks
- If the path is incorrect, the OS will not be able to locate it

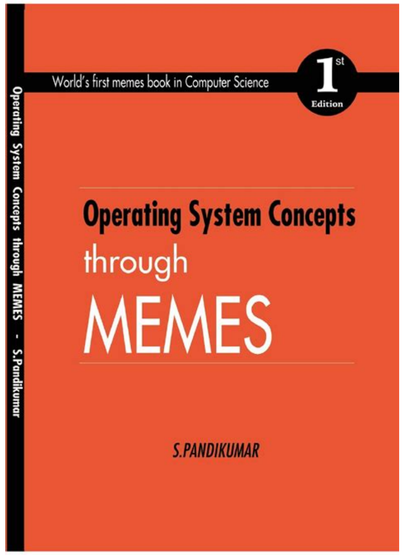
# Read from a file

given an absolute path to the file

/cis3110/textbook.png

split the address into parts

/ cis3110 textbook.png




load the superblock into RAM

locate the root inode first

inode for root directory

| |
|---|
| 2 |
| mode |
| link count |
| uid |
| gid |
| fsize |
| times |
| block ptr |

look for cis3110 in its mapping

contents of the root directory inode

| | |
|---|---|
| . | 2 |
| .. | 2 |
| cis3110 | 55663734 |
| … | … |

follow the pointer to the inode (if the directory is listed)

inode 55663734

| |
|---|
| 55663734 |
| mode |
| link count |
| uid |
| gid |
| fsize |
| times |
| block ptr |

find the file inode (if it is listed)

contents of the cis3110 directory inode

| | |
|---|---|
| . | 55663734 |
| .. | 1 |
| textbook.png | 55607504 |
| … | … |

follow the pointer to the inode and get the data

inode 55607504

| |
|---|
| 55607504 |
| mode |
| link count |
| uid |
| gid |
| fsize |
| times |
| block ptr |

block ptr to the actual image bytes on hd

# Read from a file

number of reading/writing operations to access

path=/cis3110/textbook.png

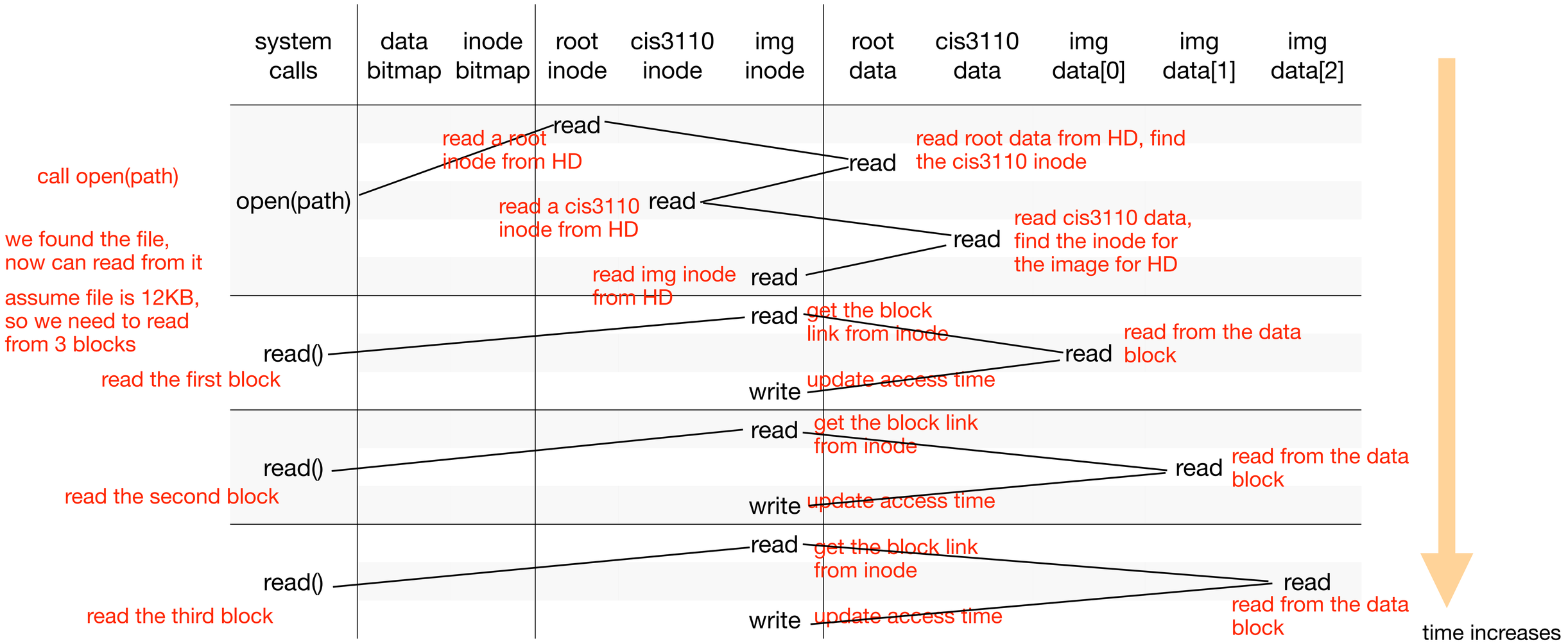
| system calls | data bitmap | inode bitmap | root inode | cis3110 inode | img inode | root data | cis3110 data | img data[0] | img data[1] | img data[2] |
|---|---|---|---|---|---|---|---|---|---|---|
| open(path) | | | read | | | | | | | |
| | | | | | | read | | | | |
| | | | | read | | | | | | |
| | | | | | | | read | | | |
| | | | | | read | | | | | |
| read() | | | | | read | | | | | |
| | | | | | | | | read | | |
| | | | | | write | | | | | |
| read() | | | | | read | | | | | |
| | | | | | | | | | read | |
| | | | | | write | | | | | |
| read() | | | | | read | | | | | |
| | | | | | | | | | | read |
| | | | | | write | | | | | |

time increases

# Read from a file

number of reading/writing operations to access

path=/cis3110/textbook.png



# Read from a file

- Reading from a file with a 3-part path stored in 3 blocks
  - **11 reads** from HD
  - **3 writes** to HD
- Number of I/O operations will **scale proportional** to
  - **path length** -> longer paths will require accessing more inodes
  - **file size** -> larger files will require accessing more data blocks

# Creating a file

number of reading/writing operations to create

path=/cis3110/textbook.pdf

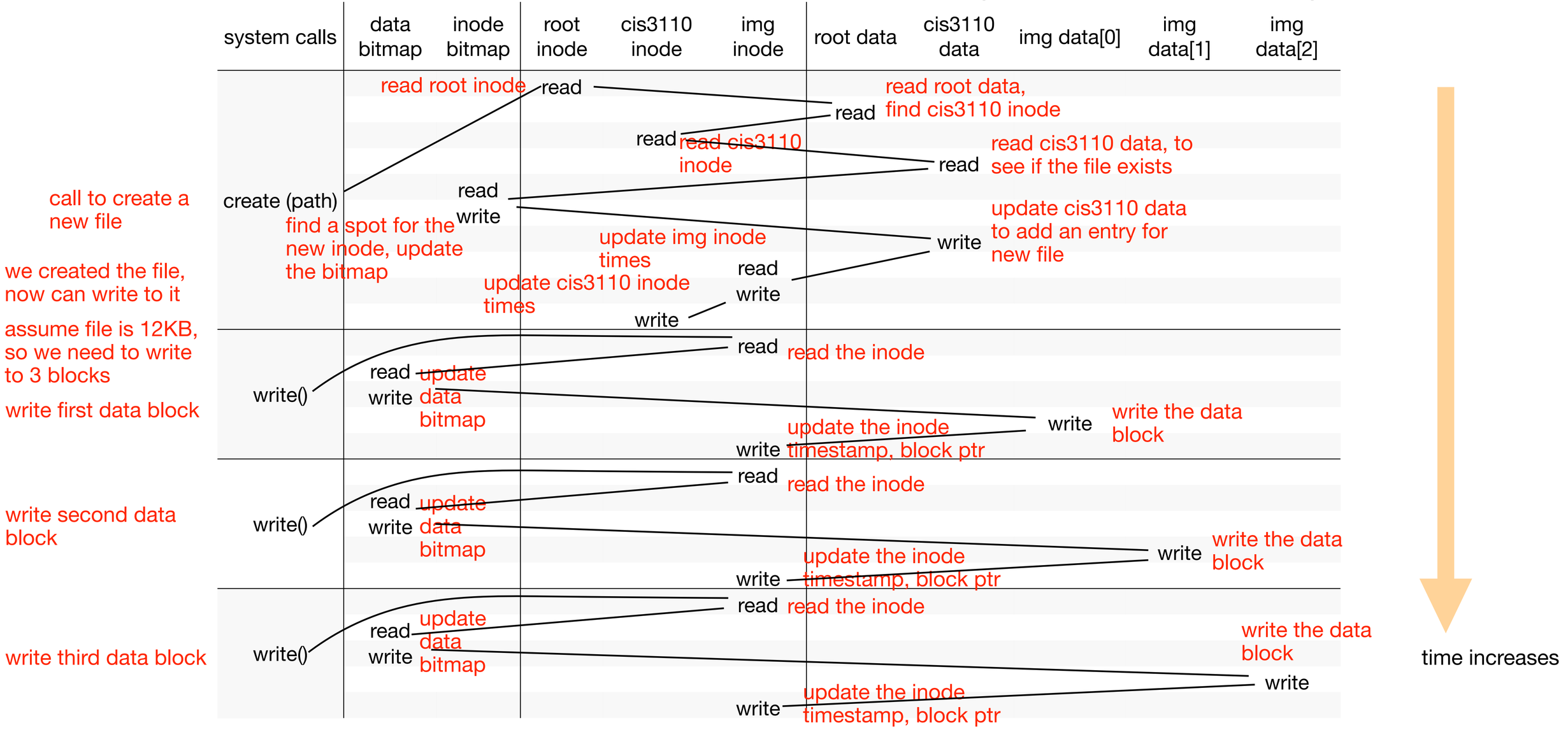
| system calls | data bitmap | inode bitmap | root inode | cis3110 inode | img inode | root data | cis3110 data | img data[0] | img data[1] | img data[2] |
|---|---|---|---|---|---|---|---|---|---|---|
| create (path) | | | read | | | | | | | |
| | | | | | | read | | | | |
| | | | | read | | | | | | |
| | | | | | | | read | | | |
| | | read | | | | | | | | |
| | | write | | | | | | | | |
| | | | | | | | write | | | |
| | | | | | read | | | | | |
| | | | | | write | | | | | |
| | | | | write | | | | | | |
| write() | | | | | read | | | | | |
| | read | | | | | | | | | |
| | write | | | | | | | | | |
| | | | | | | | | write | | |
| | | | | | write | | | | | |
| write() | | | | | read | | | | | |
| | read | | | | | | | | | |
| | write | | | | | | | | | |
| | | | | | | | | | write | |
| | | | | | write | | | | | |
| write() | | | | | read | | | | | |
| | read | | | | | | | | | |
| | write | | | | | | | | | |
| | | | | | | | | | | write |
| | | | | | write | | | | | |

time increases

# Creating a file

number of reading/writing operations to create

path=/cis3110/textbook.pdf



# Creating a file

- Creating a file with a 3-part path stored in 3 blocks
  - **12 reads** from HD
  - **13 writes** to HD
- Number of I/O operations will scale proportional to
  - **path length** -> longer paths will require accessing more inodes
  - **file size** -> larger files will require accessing more data blocks

# Optimizing I/O

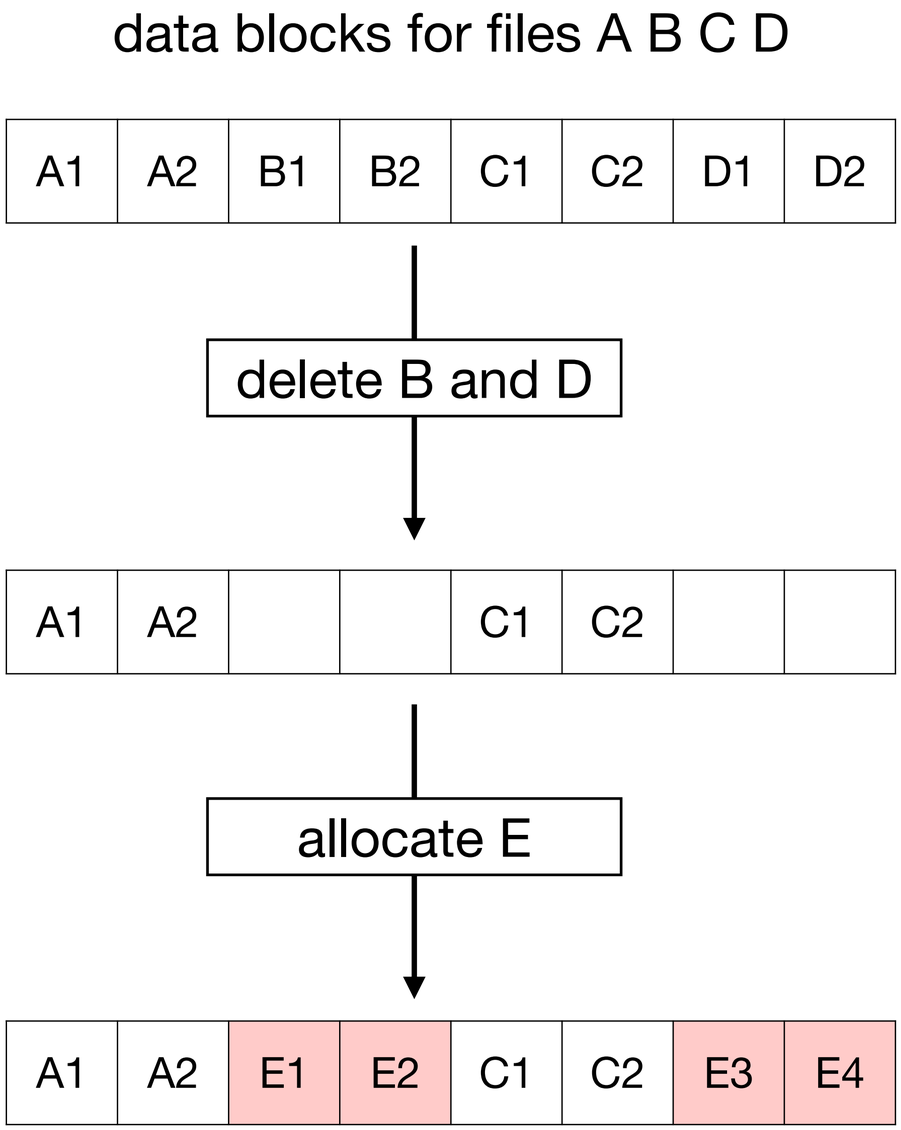
- Creating and reading are very common operations
- Problem: even simple read/write operations require **a lot of I/O**
    - files can be **anywhere** on the hard drive
    - **latencies compound** with the size of the file and length of the path
- Solution #1: **caching** in RAM
  - **unified page cache**
    - VM pages
    - inodes and data from HD
  - Helps with reads but writes still need to be done to the HD
- Solution #2: **buffering** writes to HD
  - good for performance, can use scheduling to further reduce latency
  - bad for data integrity -> updating HD may be delayed (5-30s)
- Solution #3: a **better** file system

# vsfs performance

- Very Simple File System (vsfs) is **not good**
  - Accessing and writing to files requires **a lot of I/O**
  - Only **2% of the HD bandwidth** is used, time wasted on positioning
- Problem #1: **treating HD as if it was RAM**
  - RAM (random-access memory) - cost of accessing any address is approx. constant
  - but HD has positional and transfer **latencies**
- Problem #2: **fragmentation**
  - no disk space management

# vsfs fragmentation

- vsfs does not manage free space well
- Allocations and deallocations lead to **external fragmentation**
  - file blocks are spread across the disk
- External fragmentation is **bad** for performance
  - **random I/O rate** is a lot lower than sequential (lecture #18)




E is now spread across many different locations

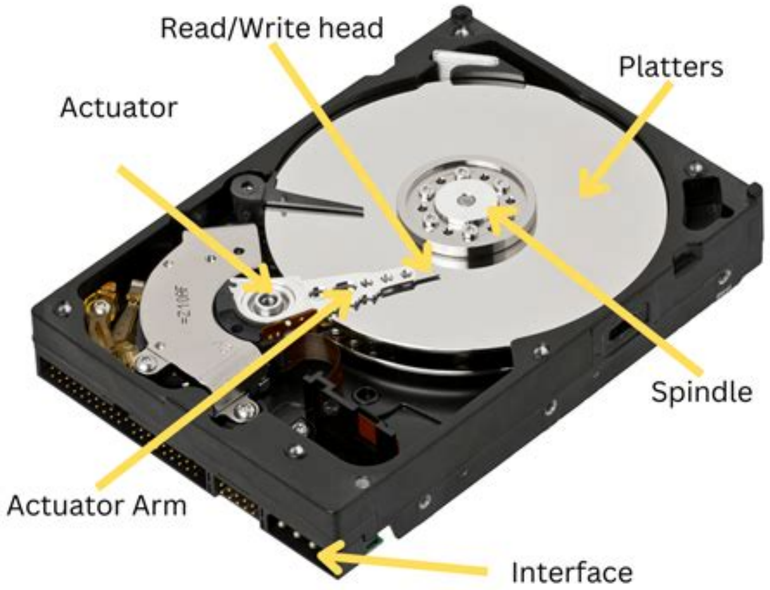
# vsfs fragmentation

- Problem: **low I/O bandwidth** due to external fragmentation
- Solution #1: **defragmentation**
  - reorganize disk data
    - make file blocks and free regions continuous
  - similar to compaction for RAM we saw earlier
  - cons: defragmentation takes forever
    - need to also update and rewrite all inodes
- Solution #2: **larger block sizes**
  - fewer blocks per file will reduce external fragmentation
  - cons: most files are small -> internal fragmentation will increase
- Solution #3: **better free space management**

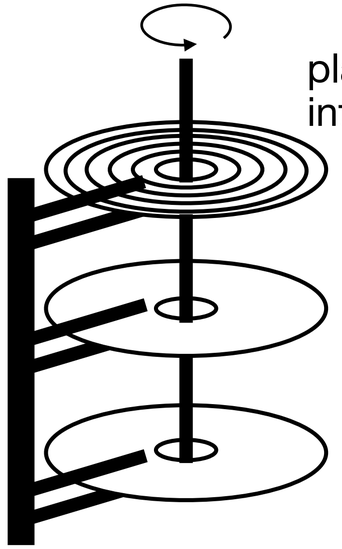
# FFS

- Fast File System (FFS)
  - **same interface as vsfs**
    - uses the same API, e.g. read(), open(), write(), close(), etc.
  - **different implementation**
    - **disk aware** data structures and allocation policies
- Key idea: **minimize positioning time** by organizing data blocks

# HD anatomy

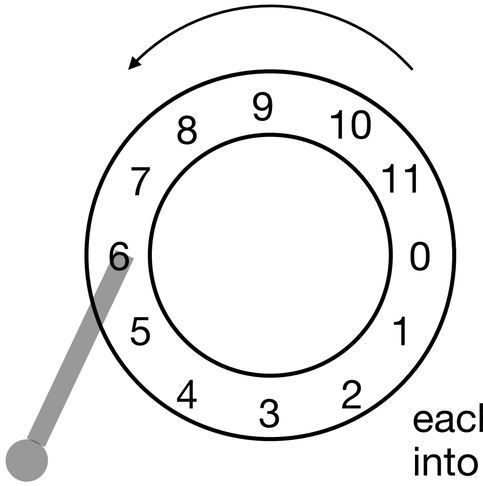



platters rotate
around the spindle



platters are split
into tracks

platters rotate CCW




each track is split
into sectors 512-
bytes each

arm with head

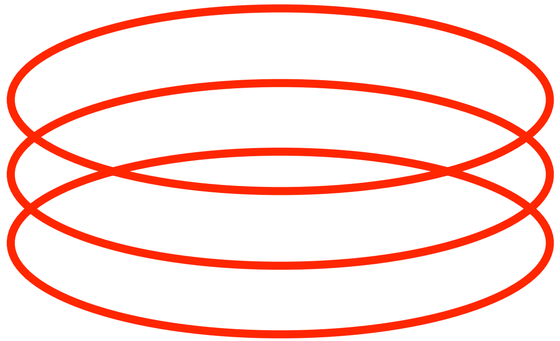
arm moves to read tracks

heads at the end of each arm read/
write onto platters on both sides

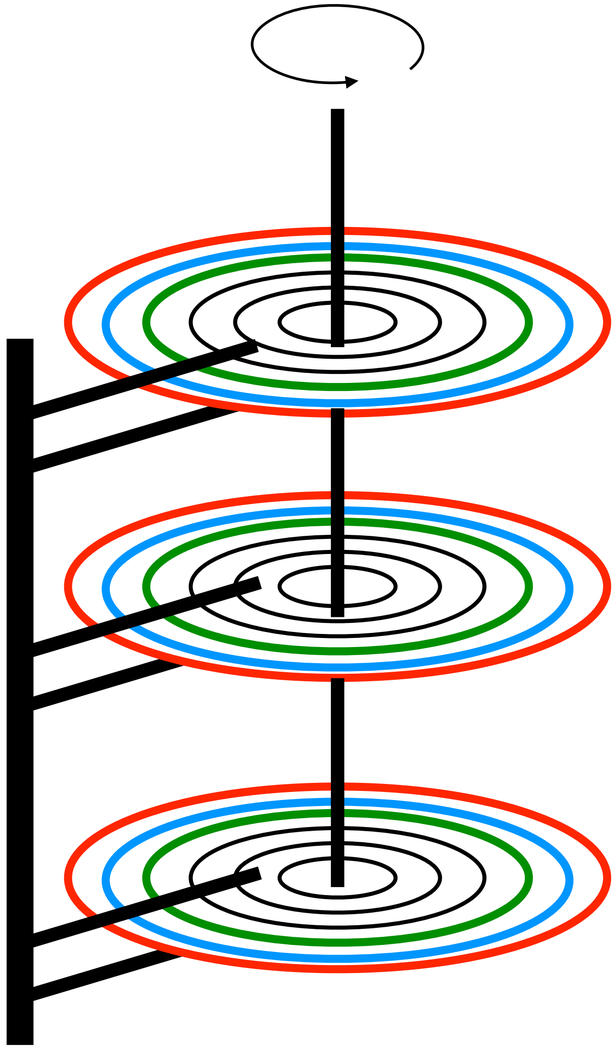
magnetizes a film of ferromagnetic
material coating the plates

# Cylinder group

- Divide hard drive into **cylinder groups**
  - "disk aware" arrangement of data blocks



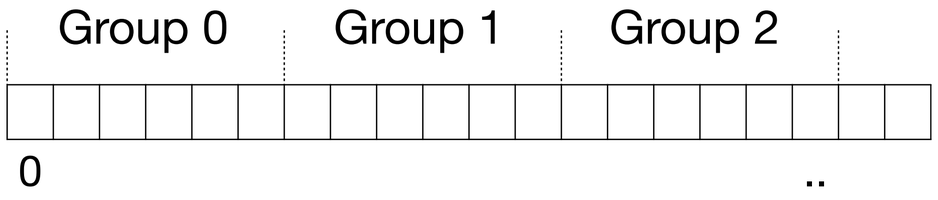
cylinder group - tracks at the
same distance from the center



same seek time for all tracks
in each cylinder group

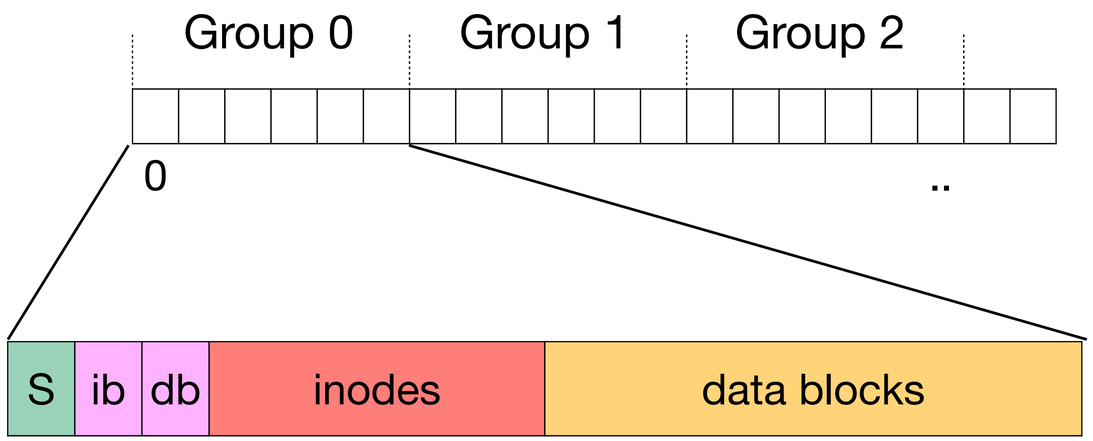
# Block group

- Problem: Most hard drives do not provide disk geometry info to the OS
- Solution: use block groups instead
  - group blocks by their disk address



# Block group

- Each block group has the following
  - a copy of the **superblock (S)**
    - speed and redundancy
  - **inode bitmap** (id) - track files
  - **data bitmap** (db) - track free space
  - **inode table** - storing file metadata
  - **data region** - storing file data




replicates vsfs structure in each block

# Allocation policy

- **Keep related stuff together and unrelated stuff apart**
- Directory placement
  - place new directory in a cylinder group with
    - few directories - to balance directories across groups
    - many free inodes - to accommodate many files
- File placement
  - data blocks and inode should be in the same group
  - files from the same directory should be in the same group
  - directory inode and data should be in the same group as their files

# Allocation policy

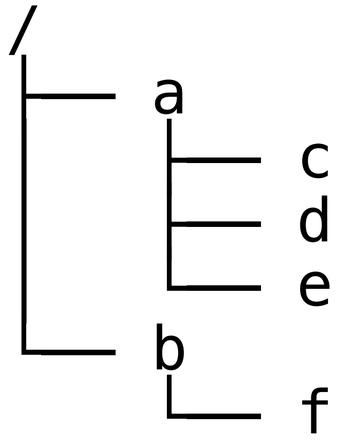
FFS allocation policy

put new directory in the emptiest group

keep directories and their files in the same group

| Group | inodes | | | | | | | | | | data | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | / | | | | | | | | | | / | | | | | | | | | |
| 1 | a | c | d | e | | | | | | | a | c | c | d | d | e | e | | | |
| 2 | b | f | | | | | | | | | b | f | f | | | | | | | |
| 3 | | | | | | | | | | | | | | | | | | | | |
| 4 | | | | | | | | | | | | | | | | | | | | |
| 5 | | | | | | | | | | | | | | | | | | | | |
| 6 | | | | | | | | | | | | | | | | | | | | |
| 7 | | | | | | | | | | | | | | | | | | | | |

allocate 3 directories and 4 files




file needs 2 data blocks

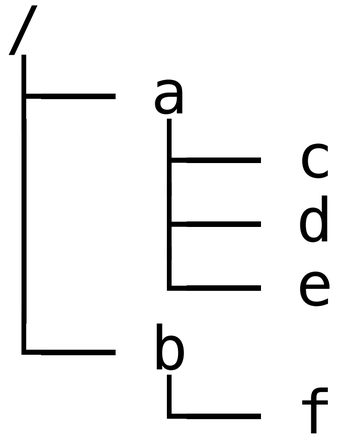
directory needs 1 data block

# Allocation policy

what if we distribute inodes across groups?

| Group | inodes | | | | | | | | | | data | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | / | | | | | | | | | | / | | | | | | | | | |
| 1 | a | | | | | | | | | | a | | | | | | | | | |
| 2 | b | | | | | | | | | | b | | | | | | | | | |
| 3 | c | | | | | | | | | | c | c | | | | | | | | |
| 4 | d | | | | | | | | | | d | d | | | | | | | | |
| 5 | e | | | | | | | | | | e | e | | | | | | | | |
| 6 | f | | | | | | | | | | f | f | | | | | | | | |
| 7 | | | | | | | | | | | | | | | | | | | | |

allocate 3 directories and 4 files




file needs 2 data blocks

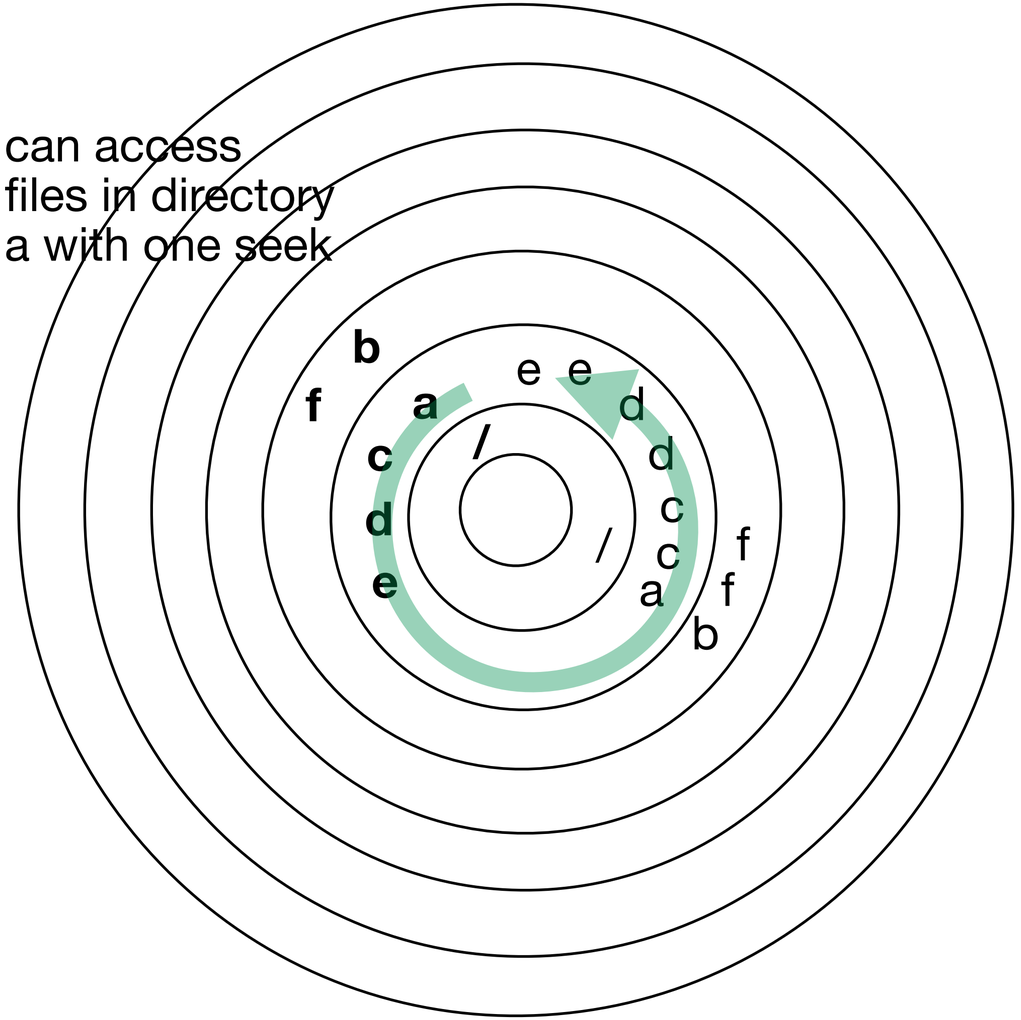
directory needs 1 data block

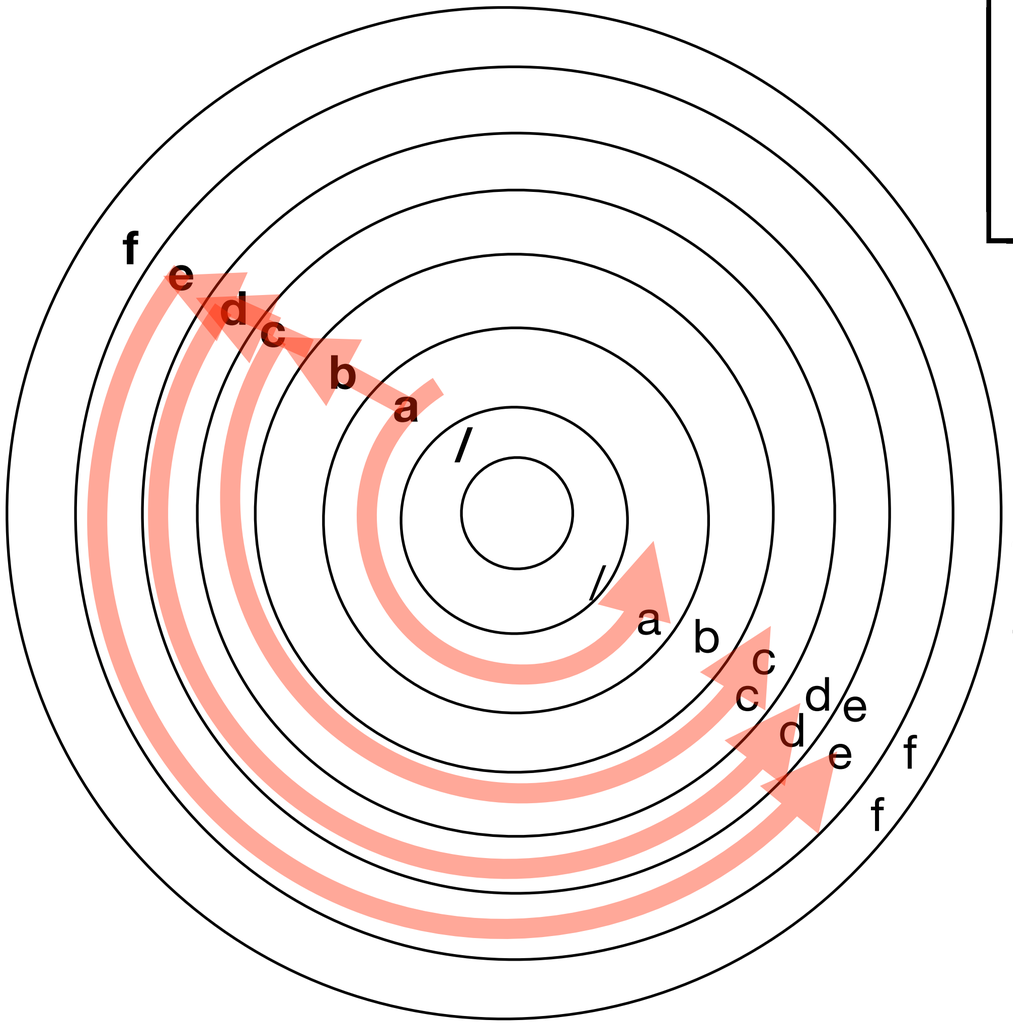
# Allocation policy

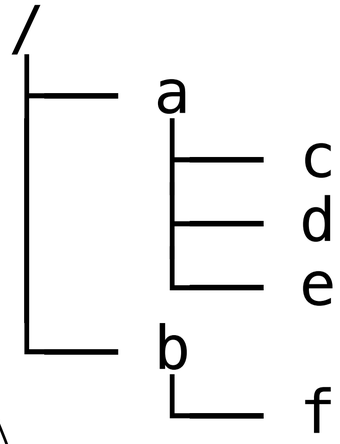
- Why FFS policy is better?
  - Takes into account **disk geometry**
    - Optimizes for contiguous access -> reduces seek time
  - Takes into account usage patterns
      - Files in the same directory are often accessed together
        - e.g. compiling an executable from files in the source directory

# Allocation policy

can access
files in directory
a with one seek




4 seeks and 4
rotations to access
files in directory a

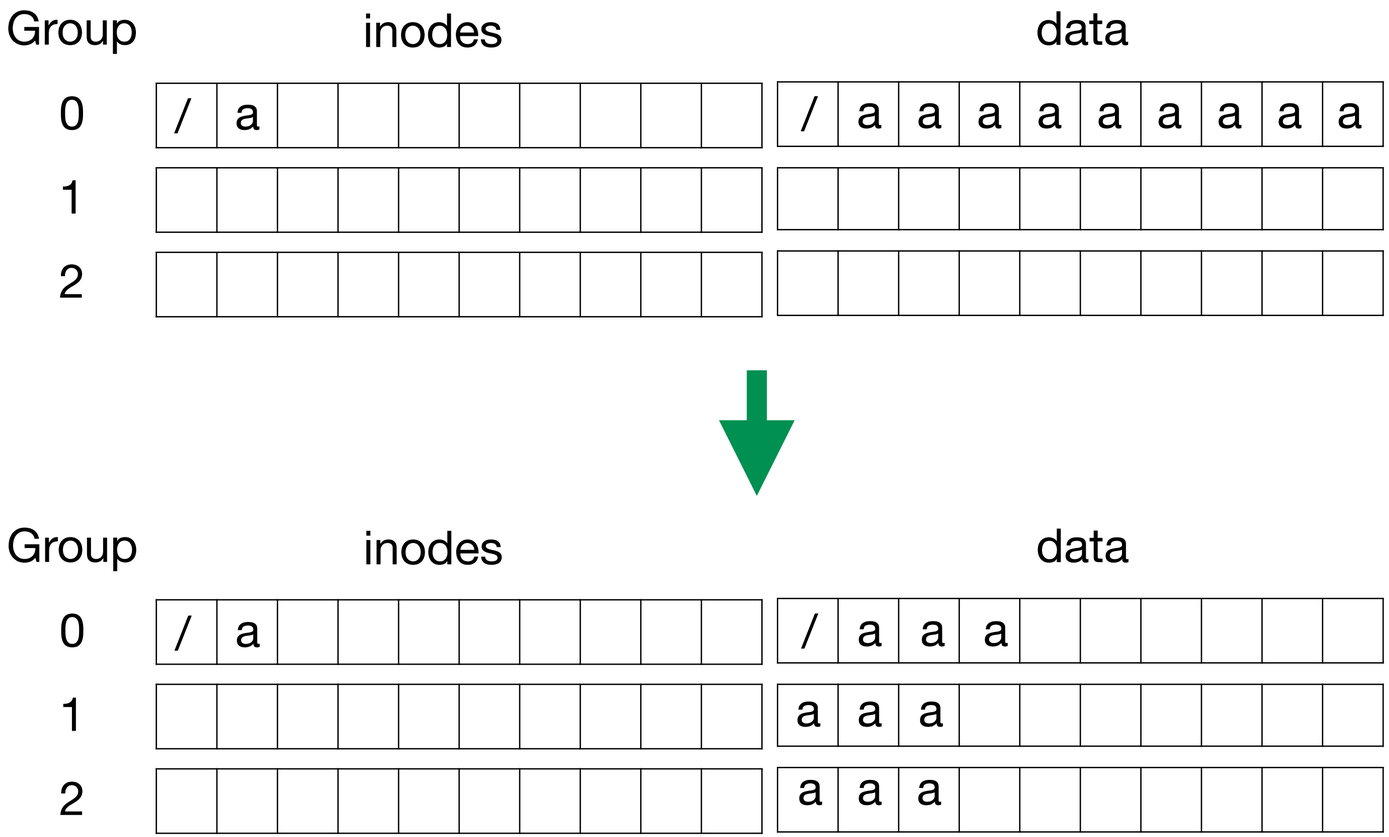
FFS policy takes hard drive geometry into
account for faster access

distributing data across hard drive will
increase positioning time

# Allocation policy

- Allocating large files
  - allocate direct blocks in the same group as the directory
  - distribute indirect blocks to different groups

what if you have a very large file?
it will fill up the group and no other files from the same directory can be here



# FFS optimizations

- **Reducing internal fragmentation**
  - most files are 2KB in size and typical block size is 4KB
    - wasting 50% of the space
  - Solution: split 4KB blocks into 512-byte sub-blocks
- Dealing with many **small writes** into sub-blocks
  - Solution: buffer them into 4KB writes
- **Long file names and symbolic links**
- **Operation for renaming files**

# UFS

- Unix file system (UFS) is a descendant of FFS
- Supported by many Unix distributions
  - FreeBSD, OpenBSD, SunOS/Solaris
- Was supported by Mac OS until 2007

# Summary

- Very simple file system (vsfs)
  - Data structures: metadata - superblock, bitmaps, inodes; data - blocks
  - Cons: poor performance because of poor allocation policy
- Fast File System (FFS)
  - Same data structures
  - Allocates data to take into account disk geometry
    - Group data and inodes into cylinder groups with the same seek time

# Reading

- Chapter 40 HDD
  - https://pages.cs.wisc.edu/~remzi/OSTEP/file-implementation.pdf
- Chapter 41 Locality and The Fast File System
  - https://pages.cs.wisc.edu/~remzi/OSTEP/file-ffs.pdf

# Quiz 18

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999